Stem-Agent

A Self-Differentiating Python QA Agent

Maël Pierron

---

1. Approach

1.1 Domain choice

The challenge invites the builder to choose any problem class. I selected Python code quality assurance for two reasons. First, it is directly relevant to JetBrains' core product space — static analysis and IDE-integrated inspections. Second, it offers a clear, reproducible evaluation: a labeled dataset of buggy files with a known ground truth, and an objective before/after comparison against an existing baseline tool (pyflakes).

1.2 Architecture

The stem agent is structured around four cooperating modules, each corresponding to a phase of differentiation:

- Explorer — given the problem class "Python code quality assurance and bug detection", queries GPT-4o to produce a structured JSON plan: typical expert approaches, relevant bug categories, detection strategies, and a suggested architecture.
- Architect — takes the Explorer's plan and generates a concrete Python class (QAAgent) using only the standard library (ast, re). On subsequent iterations, it receives structured feedback from the Evaluator and rewrites the agent accordingly.
- Evaluator — dynamically loads the generated QAAgent, runs it against a labeled dataset of 10 Python files (37 known bugs across 12 categories), and computes Recall, Precision, and F1 against the ground truth. It also produces actionable feedback: which categories were missed, how many false positives appeared on the clean file, and any runtime errors encountered.
- Loop — orchestrates the Explore → Architect → Evaluate cycle for up to 7 attempts. It stops early if Recall ≥ 70% and Precision ≥ 50%. On exhaustion, it restores the best-performing attempt.

The before/after comparison uses pyflakes as a baseline — a standard tool that catches only undefined names and unused imports, representative of what a minimal out-of-the-box linter provides.

1.3 Dataset

To ensure full reproducibility, I built a hand-crafted dataset of 10 Python files: nine containing known bugs and one clean file to measure the false positive rate. The 37 bugs span 12 categories: unused_variable, mutable_default_argument, bare_except, comparison_to_none, comparison_to_true, builtin_shadowing, resource_leak, class_shared_state, missing_return, unreachable_code, division_by_zero, and harder categories including sql_injection, logic_error, off_by_one, and type_error. Ground truth is stored in a structured JSON file loaded by the Evaluator at each iteration.

2. Experiments and Results

2.1 Before/after comparison

Table 1 — Before/after comparison: pyflakes baseline vs. best Stem-Agent output (attempt 3)

Metric	Baseline (pyflakes)	Stem-Agent (best)	Delta
Recall	11.76%	52.94%	**+41.18%**
Precision	100.00%	90.00%	**−10.00%**
F1 Score	21.05%	66.67%	**+45.62%**

The Stem-Agent achieves a +45.62-point F1 improvement over the baseline. The small precision trade-off (−10 points) is expected: pyflakes never produces false positives because it checks only a narrow set of syntactic properties, while the Stem-Agent applies broader semantic heuristics that introduce some unavoidable noise.

2.2 Iteration history

Table 2 — Metrics across all 7 attempts (target thresholds: Recall ≥ 70%, Precision ≥ 50%)

Attempt	Recall	Precision	F1	Key observation
1	58.82%	66.67%	62.50%	First generation — FP on clean file
2	52.94%	78.26%	63.16%	Stricter rules, lost some recall
3 ★	52.94%	90.00%	66.67%	Best F1 — restored as final agent
4	50.00%	89.47%	64.15%	Continued tightening
5	58.82%	66.67%	62.50%	Regressed — similar to attempt 1
6	58.82%	66.67%	62.50%	No improvement (plateau)
7	58.82%	66.67%	62.50%	Max attempts reached

3. What Surprised Us

3.1 The matching problem was invisible until execution

The first complete run produced TP=0 across every file. The Evaluator compared the generated finding types to the ground truth strings with strict equality. GPT-4o had generated names like "unused_import", "resource-leak", or "UnusedVariable" instead of the exact "unused_variable" and "resource_leak" strings specified in the ground truth. The agent was correctly finding bugs — the Evaluator simply could not see them.

The fix required a two-pronged approach: a flexible string matching function in the Evaluator (normalised lowercase, substring match, word-level overlap), and an explicit list of required type names injected into the Architect's prompt. This was a failure of interface contract design, not of the language model itself.

3.2 The feedback loop did not reliably improve recall

The iteration history reveals a troubling pattern: after a strong attempt 3 (F1=66.67%), the agent regressed in attempts 5, 6, and 7, cycling back to F1=62.50% — the same value as attempt 1. The feedback mechanism was functioning correctly; the Evaluator accurately identified missed categories and false positives. But GPT-4o at temperature=0.2 would occasionally revert to an earlier generation style, re-introducing the same false positive patterns on the clean file regardless of the feedback it received.

This suggests that prompt-injected feedback has a limited effective horizon. It works for one or two corrections but does not accumulate reliably across many iterations. A more robust mechanism would maintain a persistent, structured memory of every prior failure rather than passing only the most recent iteration's feedback.

3.3 Some bug categories are structurally unreachable with AST alone

The categories in buggy_05.py (logic_error, off_by_one) and buggy_07.py (sql_injection, style) were never detected across all 7 attempts. This is not a failure of the LLM to write good code — it reflects a fundamental limitation of static AST analysis. Logic errors require semantic understanding of the program's intent; off-by-one errors require data flow analysis; SQL injection detection requires taint tracking. None of these can be reliably implemented via pattern matching over a syntax tree.

The Evaluator consistently flagged these categories in its feedback, but the Architect could not produce a working detection because the problem is ill-posed for the chosen technique. In retrospect, the dataset should have been scoped to what is tractable for static analysis, or the system should have been capable of recognising the mismatch and escalating to a different class of tool.

---

4. What Failed

4.1 The performance threshold was never reached

Despite 7 iterations, the agent never crossed the target of Recall ≥ 70% and Precision ≥ 50% simultaneously. The best result (attempt 3) achieved Recall=52.94% with Precision=90%, meaning the agent was highly conservative — it rarely reported false positives, but it missed a large fraction of real bugs. The threshold was too ambitious given that several dataset categories (logic errors, SQL injection) are out of reach for pure AST-based analysis.

4.2 Growing constraint in the Architect's prompt reduces autonomy

To prevent GPT-4o from generating code that used non-existent AST methods (such as .next_sibling, imported from its knowledge of BeautifulSoup), the Architect's prompt was progressively enriched with a detailed skeleton specifying method signatures, forbidden imports, and partial example implementations. This improved reliability but reduced genuine self-differentiation: the Architect was no longer freely choosing its architecture — it was completing a template I had pre-designed.

This represents an honest tension at the core of the approach. A truly autonomous stem agent would require a sandboxed execution environment where generated code could be validated and corrected programmatically, rather than encoding constraints into natural language prompts. With more time, I would have implemented this execution-first loop.

4.3 No persistent memory across iterations

Each Architect call receives only the feedback from the immediately preceding attempt. There is no cumulative record of what has been tried and failed. As a result, the agent can regress — generating

code similar to a previous failed attempt — without any mechanism to detect or prevent it. A structured failure log, e.g. "in attempt 4, removing the division_by_zero check reduced recall — do not remove it", would likely have broken the plateau observed in attempts 5 through 7.

---

5. What We Would Do With More Time

- Execution-based validation: run the generated QAAgent in a subprocess before passing it to the Evaluator, catch import errors and AttributeErrors immediately, and retry without consuming an iteration budget.
- Cumulative feedback memory: maintain a structured log of every attempt's failures — missed categories, false positive patterns, runtime errors — and include the full history in each subsequent Architect prompt. This directly addresses the regression observed in attempts 5–7.
- Dataset redesign: restrict the ground truth to bug categories that are tractable for AST-based static analysis, and add more files per category to improve statistical confidence in the metrics.
- Tool discovery: allow the Explorer to reason about which existing Python tools (pylint, flake8, mypy, bandit) cover which bug categories, and let the Architect call these tools as subprocesses rather than reimplementing their logic from scratch.
- Multi-domain generalisation: run the stem agent on a second, unrelated problem class (e.g. security scanning or documentation quality) to verify that the architecture generalises without modification, which is the original biological promise of the stem cell metaphor.

Code and dataset: github.com/MaelPierron/Stem-Agent